Lab Documentation & Walkthrough

Matthew Tran

AWS Cloud Misconfiguration Detection Lab

Project Overview

In this lab, I set up a cloud security monitoring environment using AWS free tier services. My goal was to simulate two common real-world misconfigurations, detect them using AWS-native logging and threat detection tools, and then remediate them.

The two misconfigurations I simulated were:

- An S3 bucket made publicly accessible, exposing a file containing sensitive data
- An IAM user granted AdministratorAccess, violating the principle of least privilege

Tools used in this lab:

- AWS S3 - object storage, used to simulate a data exposure misconfiguration
- AWS IAM - identity and access management, used to simulate privilege escalation
- AWS CloudTrail - API activity logging, used to detect both misconfigurations
- AWS GuardDuty - threat detection service, used to review findings

Lab Objectives

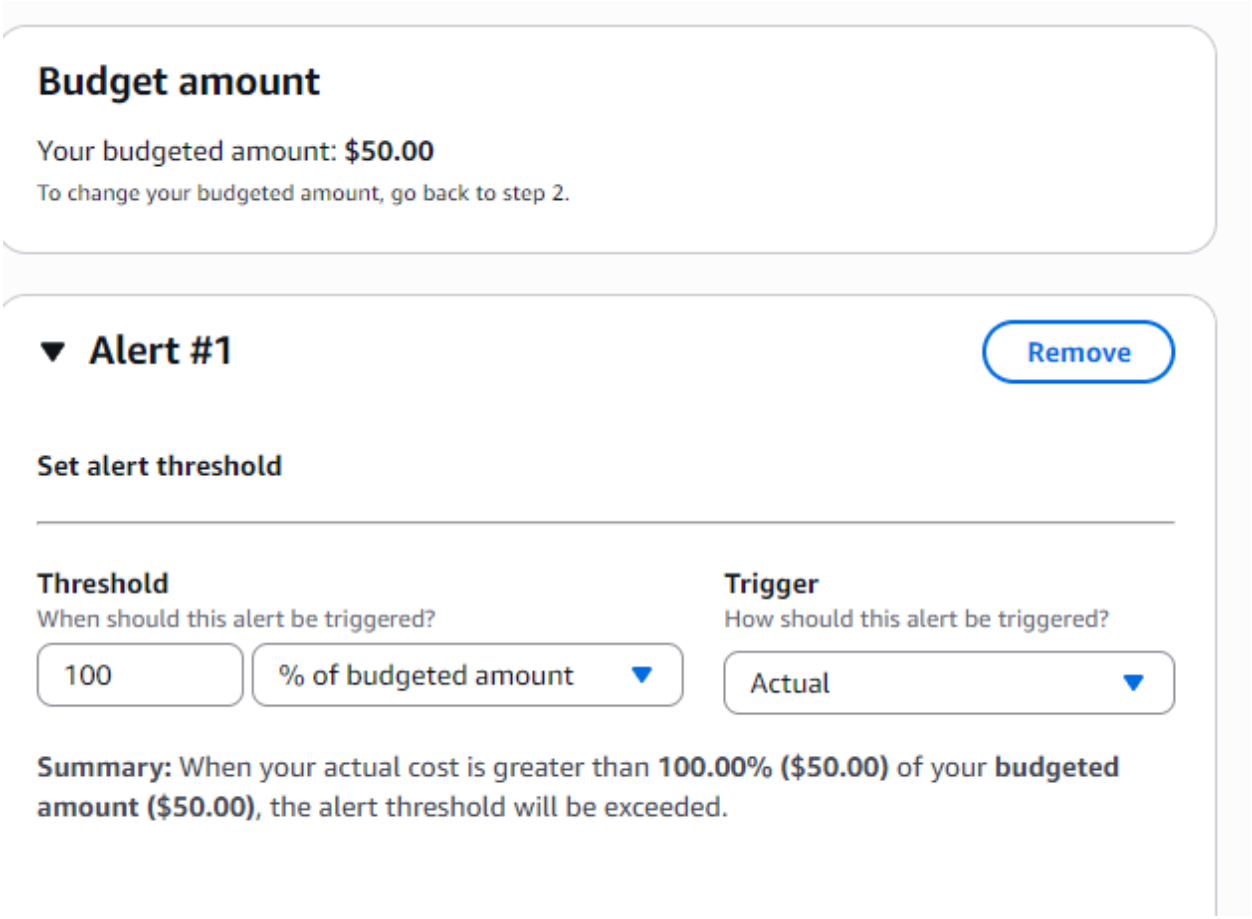
1. Create an AWS free tier account and configure a billing alert
2. Enable CloudTrail to log all API activity in my account
3. Enable GuardDuty for automated threat detection
4. Intentionally misconfigure an S3 bucket to expose it publicly
5. Intentionally create an over-privileged IAM user
6. Use CloudTrail Event History to identify both misconfigurations in the logs
7. Review GuardDuty findings dashboard for flagged activity
8. Remediate both misconfigurations and document the fixes
9. Write a findings report summarizing what I detected and how I fixed it

---

Step-by-Step Lab Walkthrough

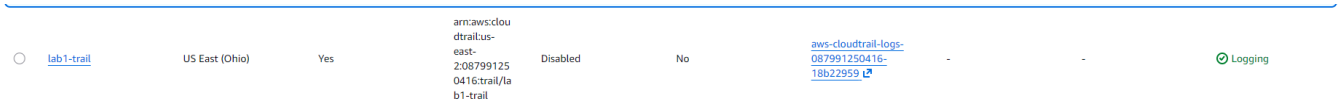
Step 1: AWS Account

I went to aws.amazon.com and created a new free tier account. As a precaution, I immediately navigated to the budget dashboard and created a billing alert set to $50.00 so I would be notified before incurring any unexpected charges. (I also made an alert for a $10 mark)



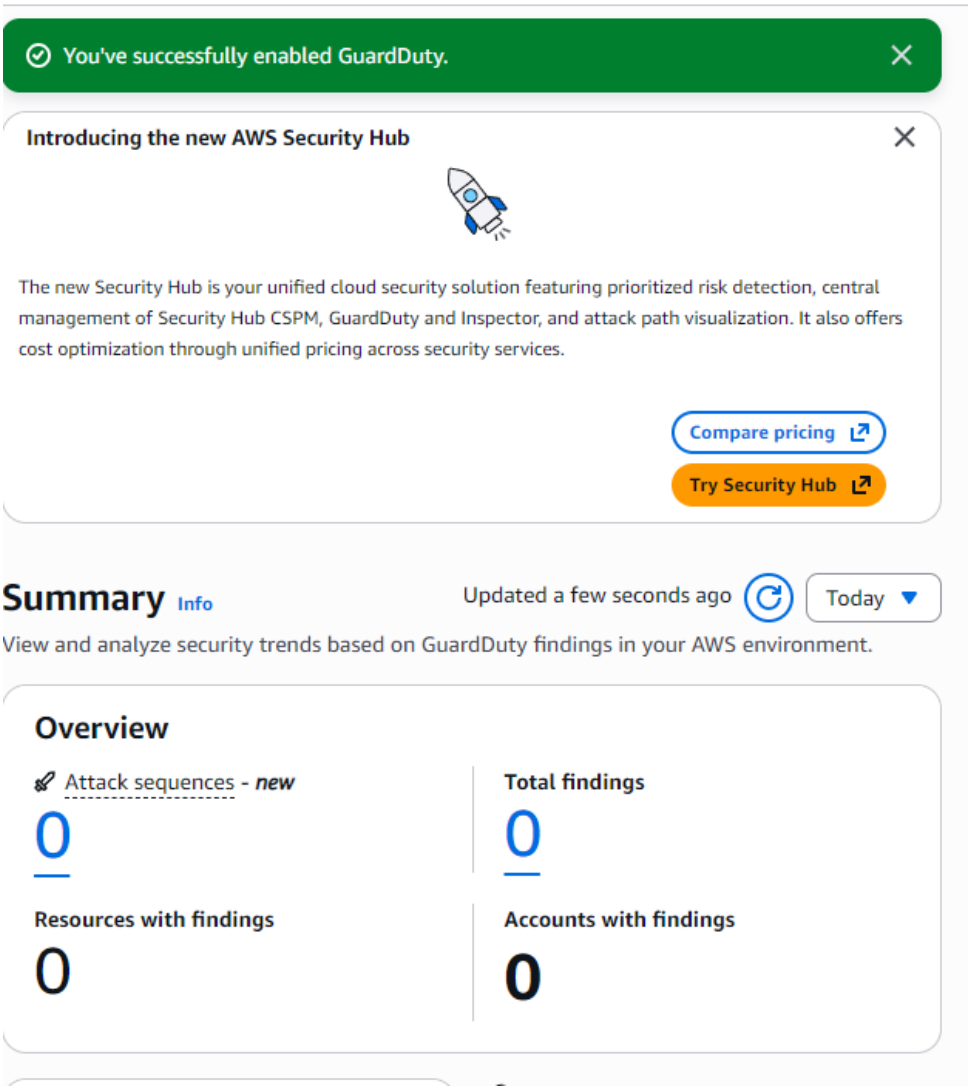
Step 2: Enable AWS CloudTrail

I navigated to the CloudTrail service in the AWS Management Console. I created a new trail and configured it to store log files in a newly created S3 bucket. Enabling CloudTrail means that every API call made in my account from this point forward is recorded, including any changes I make to S3 bucket permissions or IAM user policies.



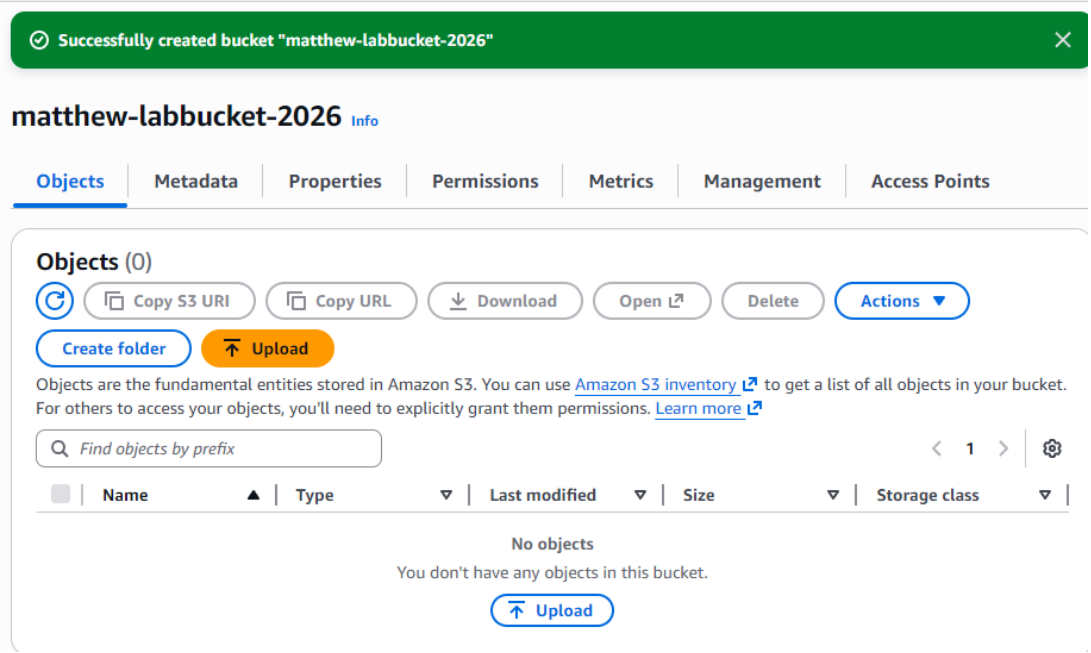
Step 3: Enable AWS GuardDuty

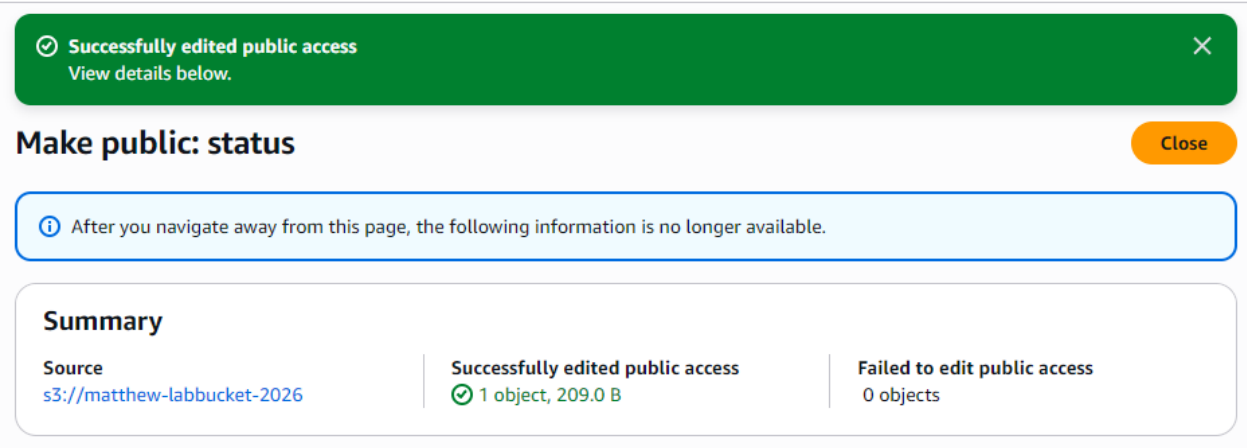
I navigated to the GuardDuty service in the AWS Console and clicked Enable GuardDuty. GuardDuty is a managed threat detection service that analyzes CloudTrail logs, VPC flow logs, and DNS logs to automatically surface suspicious activity.



Step 4: Create the S3 Misconfiguration

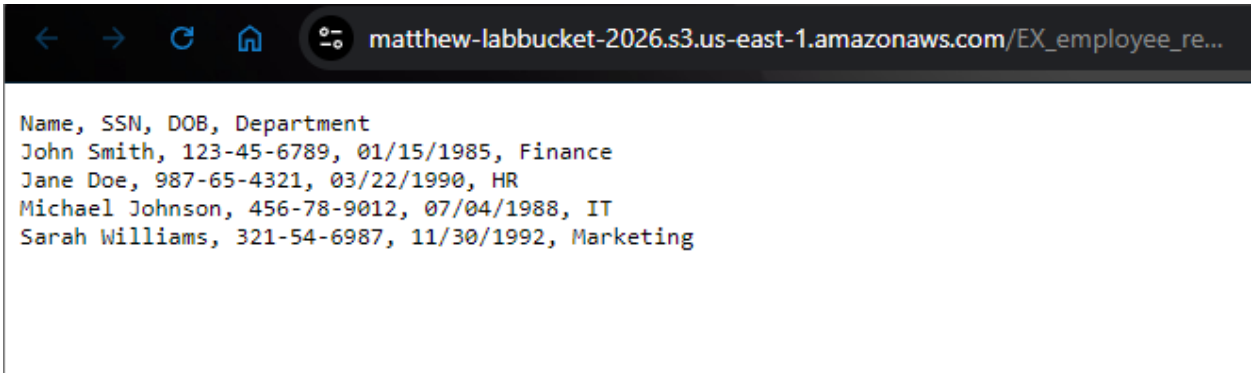
I navigated to the S3 service and created a new bucket. During setup, I disabled the Block all public access setting, which is on by default. I then created a text file on my local machine called employee_records.txt containing fake employee names and fake Social Security Numbers to simulate sensitive data. I uploaded this file to the bucket and configured it to be publicly accessible.





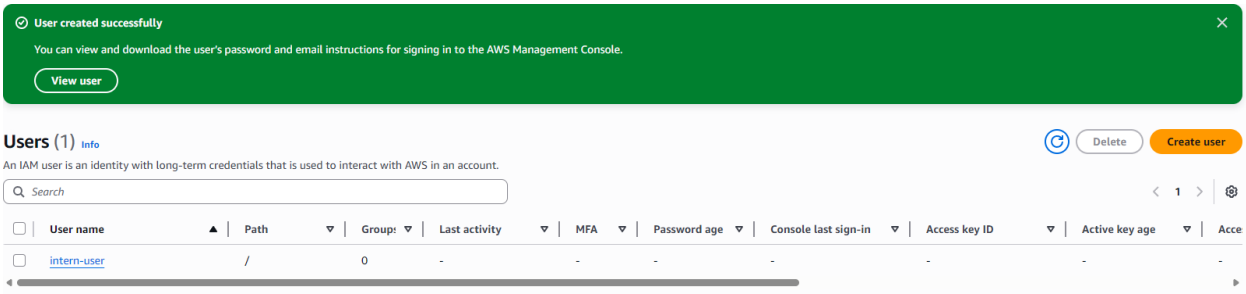
To confirm the misconfiguration worked, I copied the public URL of the file and opened it in my browser. I was able to access the contents of the file without any authentication. This simulates the type of exposure that caused the Capital One breach in 2019, where an improperly configured S3 bucket allowed unauthorized access to the personal data of over 100 million customers.

link



Step 5: Create the IAM Misconfiguration

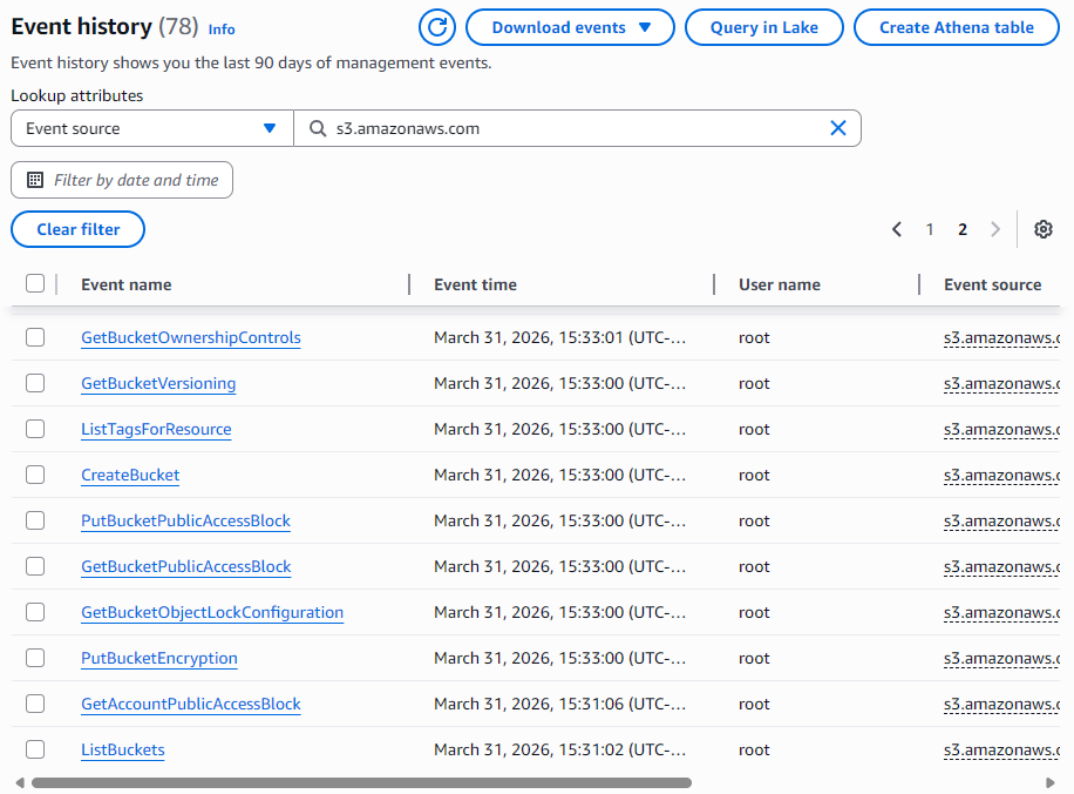
I navigated to the IAM service and created a new user named intern-user. I attached the AdministratorAccess managed policy to this user, giving a simulated intern full administrative control over my entire AWS account. This violates the principle of least privilege, which states that users should only be granted the minimum level of access required to perform their job functions.



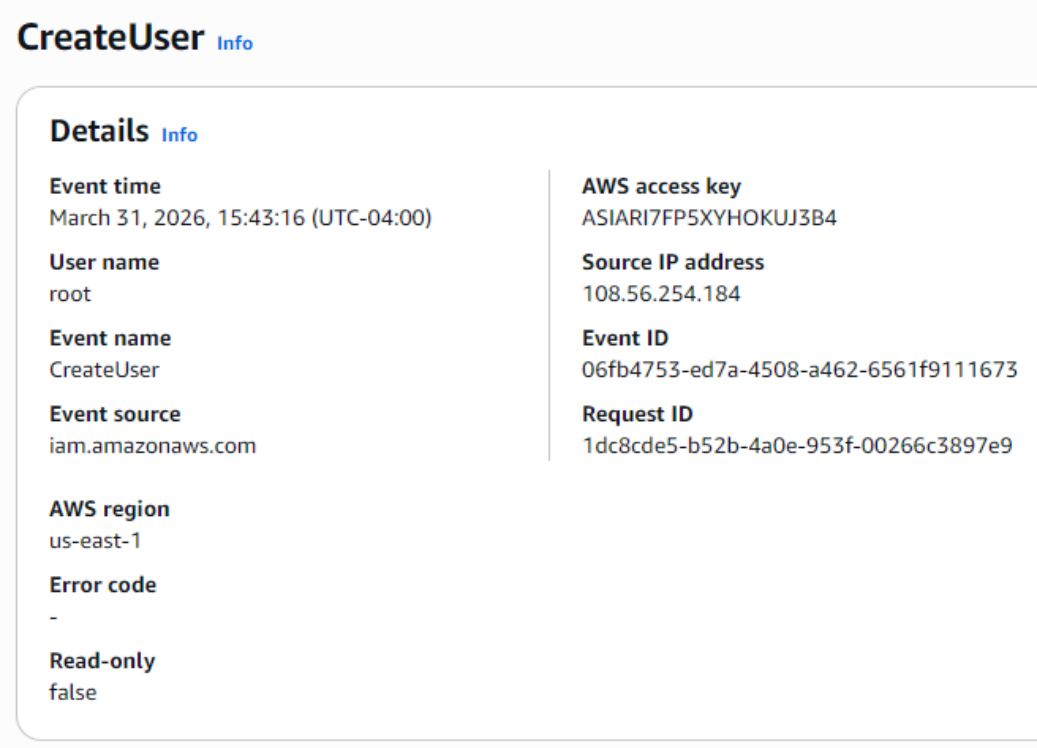
Step 6: Detect the Misconfigurations Using CloudTrail

I navigated to CloudTrail and opened the Event History view, which shows a real-time log of all API calls made in my account. I searched for the following events:

- PutBucketAcl - logged when I changed the S3 bucket's permissions to allow public access
- CreateUser - logged when I created the intern-user IAM account



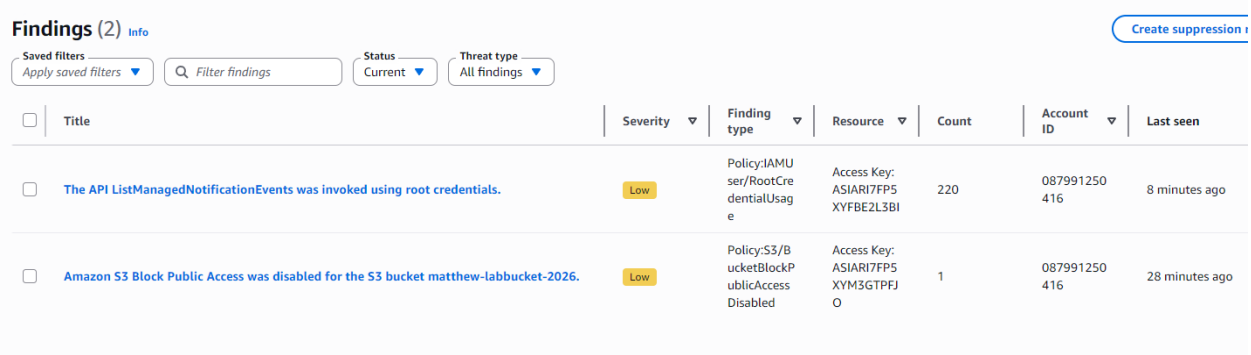

```
    },
    "eventTime": "2026-03-31T19:33:00Z",
    "eventSource": "s3.amazonaws.com",
    "eventName": "PutBucketPublicAccessBlock",
    "awsRegion": "us-east-1",
    "sourceIPAddress": "108.56.254.184",
    "userAgent": "[Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.
    "requestParameters": {
        "publicAccessBlock": "",
        "bucketName": "matthew-labbucket-2026",
        "PublicAccessBlockConfiguration": {
            "xmlns": "http://s3.amazonaws.com/doc/2006-03-01/",
            "RestrictPublicBuckets": false,
            "BlockPublicPolicy": false,
            "BlockPublicAcls": false,
            "IgnorePublicAcls": false
```




```
},
"eventTime": "2026-03-31T19:43:16Z",
"eventSource": "iam.amazonaws.com",
"eventName": "CreateUser",
"awsRegion": "us-east-1",
"sourceIPAddress": "108.56.254.184",
"userAgent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.3
"requestParameters": {
    "userName": "intern-user"
},
"responseElements": {
    "user": {
        "path": "/",
        "userName": "intern-user",
        "userId": "AIDARI7FP5XYCMP5F4S3Z",
        "arn": "arn:aws:iam::087991250416:user/intern-user",
        "createDate": "Mar 31, 2026, 7:43:16 PM"
    }
```


Step 7: Review GuardDuty Findings

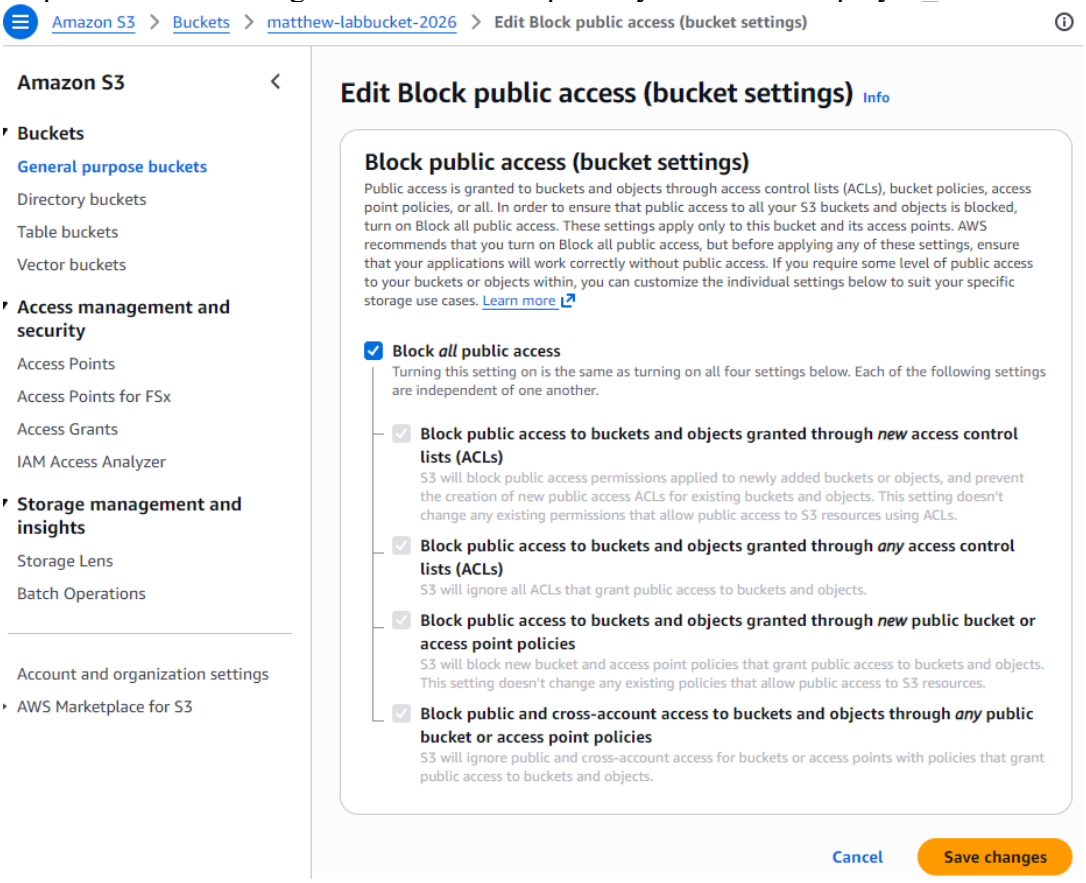
I navigated to the GuardDuty service and checked the Findings dashboard. If no findings had fired organically during the lab, I went to Settings and selected Generate Sample Findings to populate the dashboard with realistic example alerts. I reviewed the findings and noted the severity ratings, finding types, and the AWS resources involved.

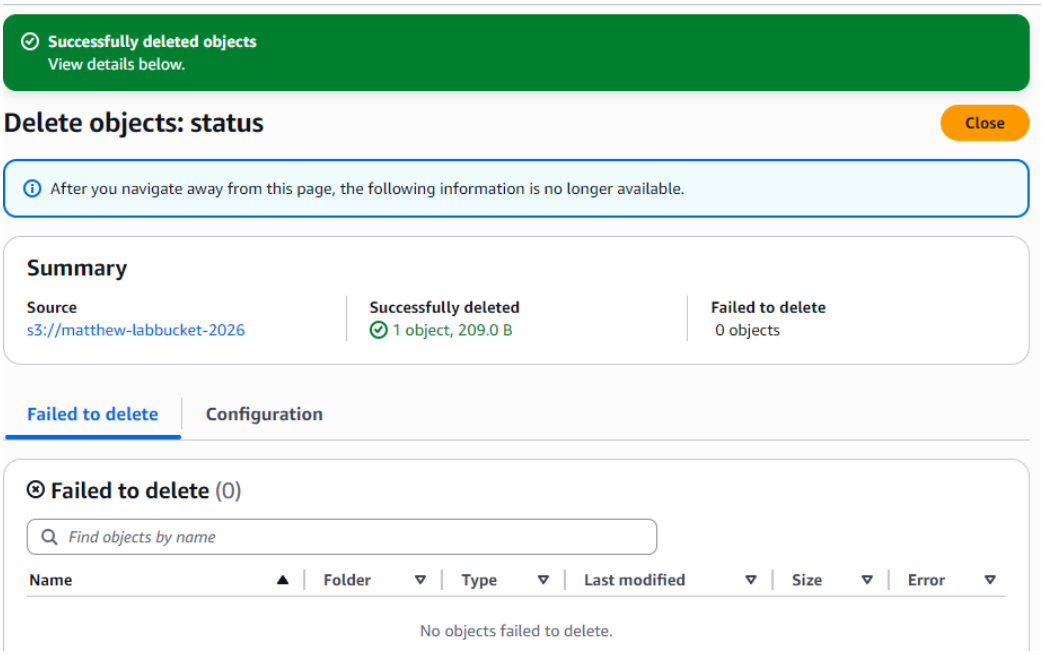


Step 8: Remediate Both Misconfigurations

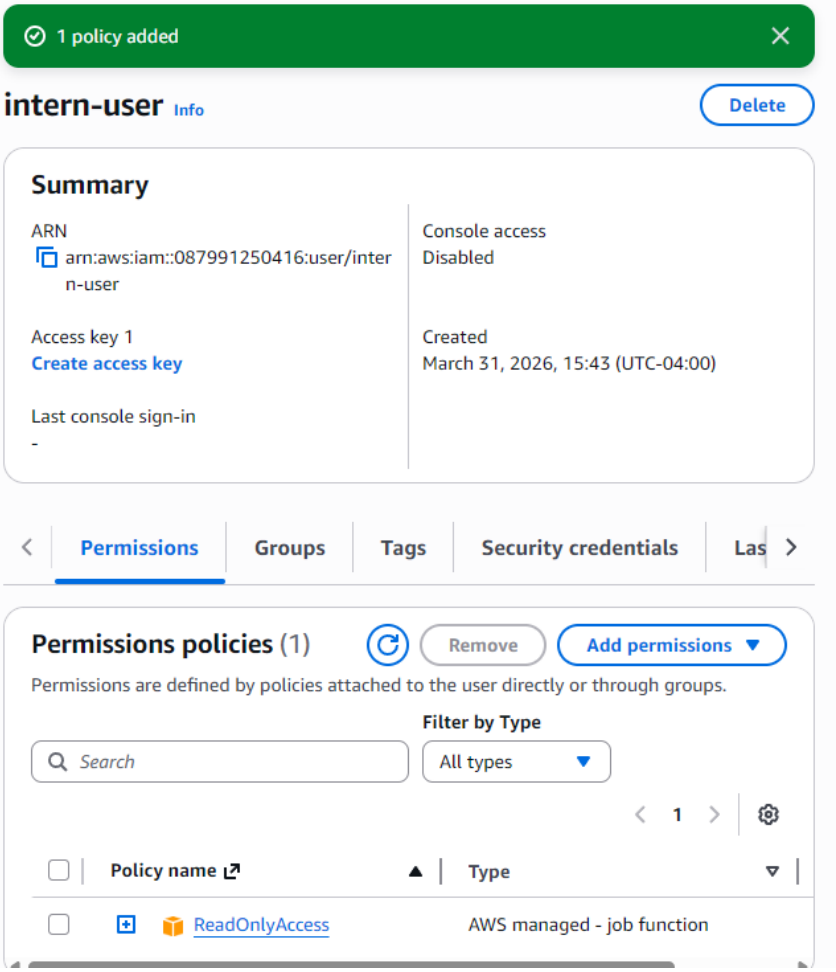
After documenting the detections, I remediated both issues:

S3 Fix: I navigated back to the S3 bucket, opened the Permissions tab, and re-enabled the Block all public access setting. I also deleted the publicly accessible employee_records.txt file.

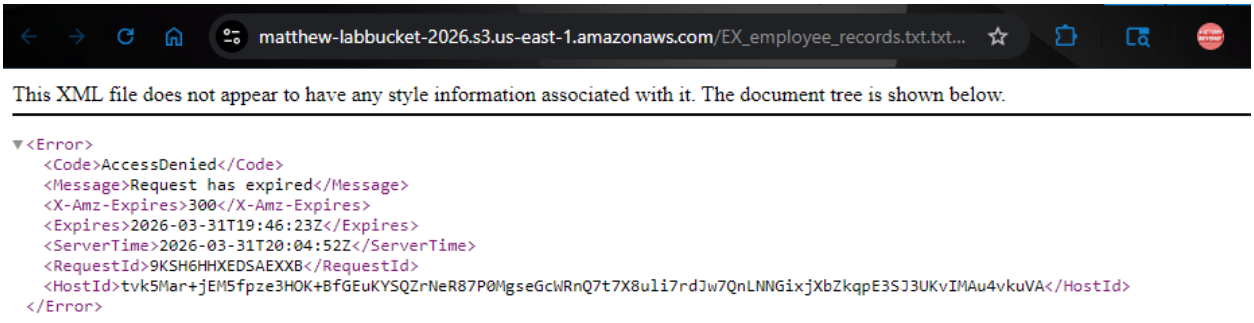




IAM Fix: I navigated back to the IAM service, opened the intern-user account, detached the AdministratorAccess policy, and replaced it with the ReadOnlyAccess managed policy.



I verified both fixes by attempting to access the S3 file URL again (access denied) and reviewing the intern-user permissions in the IAM console.



Findings Report

Executive Summary

During this lab exercise, I introduced two cloud misconfigurations into an AWS free tier environment and then detected and remediated them using AWS-native tools. Both misconfigurations are representative of real-world vulnerabilities commonly found in cloud environments during security assessments.

Finding 1: Publicly Accessible S3 Bucket

Misconfiguration: I disabled the Block all public access setting on an S3 bucket and uploaded a file containing simulated sensitive employee data. The file was accessible to anyone on the internet without authentication.

Detection: CloudTrail logged a PutBucketAcl event at the time the bucket was made public. The event record included the bucket name, the time of the change, and the user identity that made the change.

Business Risk: An exposed S3 bucket can lead to a large-scale data breach. The Capital One breach in 2019 was caused by a similar misconfiguration and exposed the personal data of over 100 million customers. In a regulated environment, this type of exposure would likely trigger HIPAA, PCI DSS, or state-level breach notification requirements.

Remediation: I re-enabled the Block all public access setting on the bucket and deleted the exposed file. Access was verified as denied upon attempting to reload the public file URL.

Finding 2: Over-Privileged IAM User

Misconfiguration: I created an IAM user named intern-user and attached the AdministratorAccess policy, granting full administrative control over the entire AWS account.

Detection: CloudTrail logged a CreateUser event when the account was created and an AttachUserPolicy event when AdministratorAccess was assigned. Both events were visible in the CloudTrail Event History.

Business Risk: An over-privileged user account creates a significant attack surface. If the intern-user credentials were compromised through phishing or credential stuffing, an attacker would have unrestricted access to all AWS services, data, and infrastructure. This violates the principle of least privilege, a foundational control in NIST CSF, CIS Controls, and ISO 27001.

Remediation: I detached the AdministratorAccess policy and replaced it with ReadOnlyAccess, limiting the user to view-only permissions. The change was reflected immediately in the IAM console.

Recommendations

- Enforce MFA on all IAM users, especially those with elevated permissions
- Enable AWS Config to continuously monitor resource configurations and alert on drift from a secure baseline
- Apply S3 Block Public Access at the account level rather than per-bucket to prevent accidental exposure
- Implement IAM Access Analyzer to automatically flag overly permissive policies
- Review CloudTrail logs regularly using automated alerting to detect configuration changes in real time

Frameworks Referenced

- NIST CSF 2.0 - Protect (PR.AC), Detect (DE.CM)
- CIS Controls v8 - Control 3 (Data Protection), Control 5 (Account Management)
- AWS Well-Architected Framework - Security Pillar

Reflection

This lab gave me hands-on experience with two of the most common causes of cloud security incidents: misconfigured storage buckets and over-privileged user accounts. I was able to see how quickly a misconfiguration can be introduced and how CloudTrail provides an audit trail that makes detection possible after the fact.

The most important takeaway for me was understanding how GuardDuty and CloudTrail work together. CloudTrail captures every API call as a raw log, while GuardDuty applies machine learning and threat intelligence to surface the most important events automatically. In a real SOC environment, analysts use both tools together to triage and investigate cloud security incidents.